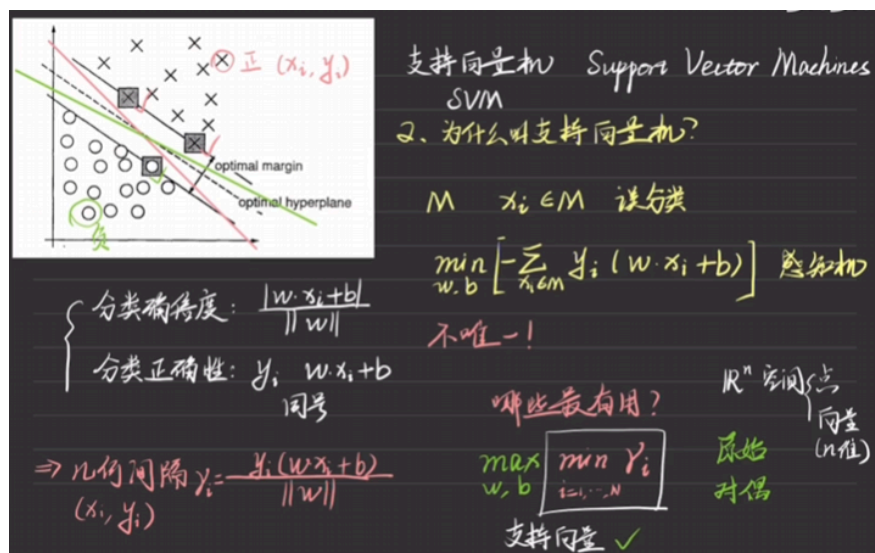


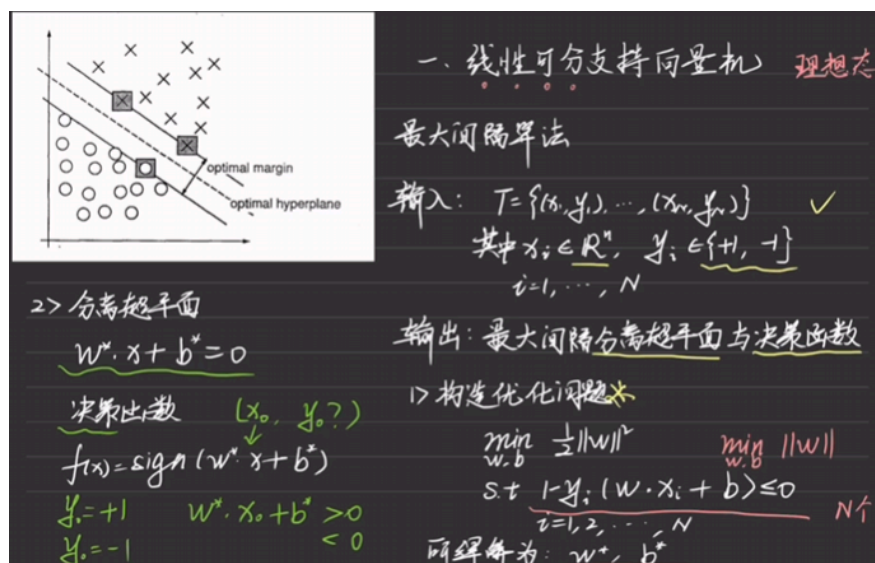
支持向量机

支持向量机的一些基本情况



可以证明最优解唯一且一定存在，证明过程略。

基本算法



上述基本算法很难实现，如何进行优化

对偶问题

内部极小化

一. 线性可分支持向量机 —— 对偶问题

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^N \alpha_i y_i (w \cdot x_i + b) + \sum_{i=1}^N \alpha_i$$

$$\text{对偶问题: } \max_{\alpha} \min_{w, b} L(w, b, \alpha)$$

① 内部极小化 $\theta_0(\alpha)$

$$w, b: \begin{cases} \nabla_w L = \frac{1}{2} \times 2w - \sum_{i=1}^N \alpha_i y_i x_i = w - \sum_{i=1}^N \alpha_i y_i x_i = 0 \\ \nabla_b L = - \sum_{i=1}^N \alpha_i y_i = 0 \end{cases}$$

$$\Rightarrow \begin{cases} w = \sum_{i=1}^N \alpha_i y_i x_i \\ \sum_{i=1}^N \alpha_i y_i = 0 \end{cases}$$

$$\theta_0(\alpha) = -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) + \sum_{i=1}^N \alpha_i$$

外部极大化

一. 线性可分支持向量机 —— 对偶问题

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^N \alpha_i y_i (w \cdot x_i + b) + \sum_{i=1}^N \alpha_i$$

$$\text{对偶问题: } \max_{\alpha} \min_{w, b} L(w, b, \alpha)$$

② 外部极大化 $\theta_0(\alpha) = -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) + \sum_{i=1}^N \alpha_i$

$$\max_{\alpha} \theta_0(\alpha)$$

$$\text{s.t. } \sum_{i=1}^N \alpha_i y_i = 0$$

$$\alpha_i \geq 0, i=1, 2, \dots, N$$

$$\alpha^* \Rightarrow w^*, b^*$$

此时得到 α ，下面讲述如何通过 α 得到 w, b 。

原始问题与对偶问题

广义拉格朗日函数:

$$L(x, \alpha, \beta) = f(x) + \sum_{i=1}^k \alpha_i c_i(x) + \sum_{j=1}^l \beta_j h_j(x)$$

Theorem (KKT 条件)

若函数 $f(x)$ 和 $c_i(x)$ 是凸函数， $h_j(x)$ 是仿射函数；并且假设不等式约束 $c_i(x)$ 是严格可行的，则 x^* 是原始问题的解， α^*, β^* 是对偶问题的解的充分必要条件是

$\nabla_x L(x^*, \alpha^*, \beta^*) = 0$	
$\alpha_i^* c_i(x^*) = 0, \checkmark$	$i = 1, 2, \dots, k$
$c_i(x^*) \leq 0, \checkmark$	$i = 1, 2, \dots, k$
$\alpha_i^* \geq 0, \checkmark$	$i = 1, 2, \dots, k$
$h_j(x^*) = 0, \checkmark$	$j = 1, 2, \dots, l$

$$1 - y_j (w^* \cdot x_j + b^*) = 0$$

$$\Rightarrow b^* = \frac{1}{y_j} - w^* \cdot x_j$$

$$= \frac{1}{y_j} - \sum_{i=1}^N \alpha_i^* y_i (x_i \cdot x_j)$$

$$y_j + 1, -1 \quad y_j = \frac{1}{y_j}$$

$$\checkmark b^* = y_j - \sum_{i=1}^N \alpha_i^* y_i (x_i \cdot x_j)$$

超平面:

$$w^* \cdot x + b^* = 0$$

决策函数

$$\text{sign}(w^* \cdot x + b^*)$$

$$\nabla_w L = w^* - \sum_{i=1}^N \alpha_i^* y_i x_i = 0$$

$$\Rightarrow w^* = \sum_{i=1}^N \alpha_i^* y_i x_i \checkmark$$

$b^* ? \quad \alpha^* = (0, 0, \dots, 0)^T \Rightarrow w^* = 0 \times$

肯定存在 $\alpha_j^* > 0 \quad \alpha_j^* y_j (w^*, b^*) = 0$

可以看出满足对应的 α 非零的 $(x_i^{(1)}, x_i^{(2)})$ 就是支持向量。

线性支持向量机

相比于最开始的算法，由硬间隔变换为软间隔。

原始问题

二. 线性支持向量机

训练集: $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$

其中 $x_i \in X \in \mathbb{R}^n$, $y_i \in Y \in \{+1, -1\}$

目标函数: $\min \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$

当 C 大时, 对误分类惩罚大
 C 小

此时 $\xi_i \in (0, 1)$ 位于间隔内
 $\xi_i > 1$ 位于间隔外 (早)

线性支持向量机, 软间隔

$y_i (w \cdot x_i + b) + \xi_i \geq 1$ ✓
 松弛变量

其中系数 C 的作用是对误分类的惩罚设置大小。

对偶问题

先来看一个对比

二. 线性支持向量机

训练集: $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$

其中 $x_i \in X \in \mathbb{R}^n$, $y_i \in Y \in \{+1, -1\}$

优化问题:

原始: $\min_{w, b, \xi_i} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$
 s.t. $y_i (w \cdot x_i + b) + \xi_i \geq 1$
 $\xi_i \geq 0, i=1, \dots, N$

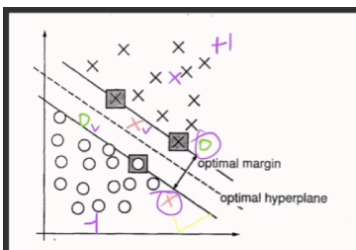
对偶: $\max_{\alpha, \mu} L$

广义拉格朗日:

$L(w, b, \xi_i, \alpha, \mu) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i + \sum_{i=1}^n \alpha_i (1 - \xi_i - y_i (w \cdot x_i + b)) - \sum_{i=1}^n \mu_i \xi_i$

实际的求解方法:

内部极小化 $\rightarrow$ 外部极大化



二. 线性支持向量机

训练集:

$$T = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

$$\text{其中 } x_i \in X \in \mathbb{R}^n, y_i \in Y \in \{+1, -1\}$$

内部极小化: $\theta_0(\alpha, \mu)$

$$L = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j)$$

$$+ \sum_{i=1}^n \alpha_i$$

广义拉格朗日:

$$L(w, b, \xi_i, \alpha, \mu) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i + \sum_{i=1}^n \alpha_i (1 - \xi_i - y_i (w \cdot x_i + b)) - \sum_{i=1}^n \mu_i \xi_i$$

$$w = \sum_{i=1}^n \alpha_i y_i x_i \quad \begin{cases} \frac{\partial L}{\partial w} = w - \sum_{i=1}^n \alpha_i y_i x_i = 0 \\ \frac{\partial L}{\partial b} = -\sum_{i=1}^n \alpha_i y_i = 0 \\ \frac{\partial L}{\partial \xi_i} = C - \alpha_i - \mu_i = 0 \end{cases}$$

$$\sum_{i=1}^n \alpha_i y_i = 0 \quad \checkmark$$

$$C - \alpha_i - \mu_i = 0 \quad (i=1, \dots, N)$$

对偶: $\max_{\alpha, \mu} \min_{w, b, \xi_i} L$

算法流程

二. 线性支持向量机

输入:

训练集:

$$T = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

$$\text{其中 } x_i \in X \in \mathbb{R}^n, y_i \in Y \in \{+1, -1\}$$

输出:

分离超平面与分类决策函数

算法:

1. 给定惩罚参数 $C > 0$, 构造优化问题

$$\min_w \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \frac{1}{2} \sum_{i=1}^n \alpha_i$$

$$\text{s.t. } \sum_{i=1}^n \alpha_i y_i = 0$$

$$0 \leq \alpha_i \leq C$$

$$i=1, \dots, N$$

2. 求解优化问题, 得到

最优解

$$\alpha^* = (\alpha_1^*, \alpha_2^*, \dots, \alpha_n^*)^T$$

3. 根据 α^* 得到

$$w^* = \sum_{i=1}^n \alpha_i^* y_i x_i$$

挑出符合 $0 < \alpha_i^* < C$ 的点

(x_j, y_j) 计算

$$b^* = y_j - \sum_{i=1}^n \alpha_i^* y_i (x_i \cdot x_j)$$

4. 分离超平面: $w^* \cdot x + b^* = 0$

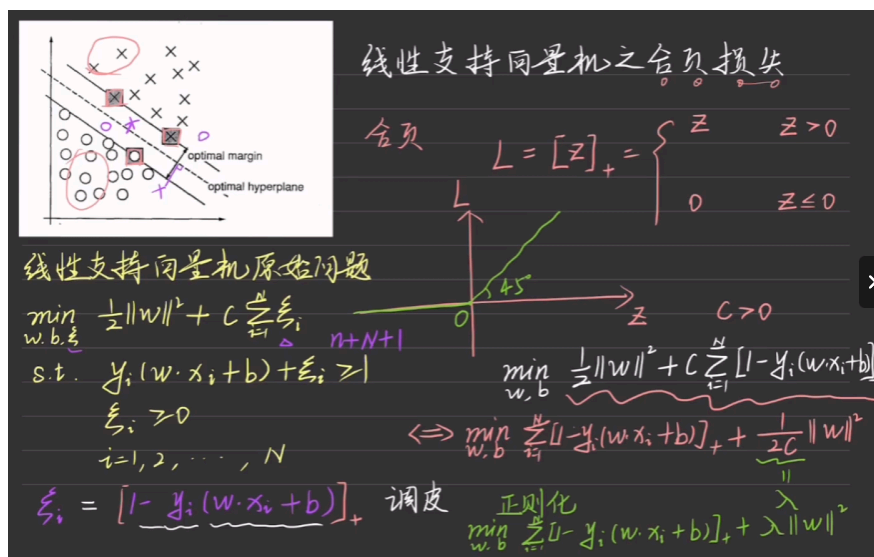
决策函数: $f(x) = \text{sign}(w^* \cdot x + b^*)$

同时需要注意的是, 此时的支持向量不再仅仅是边界上的点了。

如果 $0 < \alpha < C \rightarrow \epsilon = 0$, 此时的支持向量在边界上。

如果 $\alpha = C \rightarrow \epsilon > 0$, 即此时的支持向量是间隔内的点, 超平面上的点和另一边的误分类点。

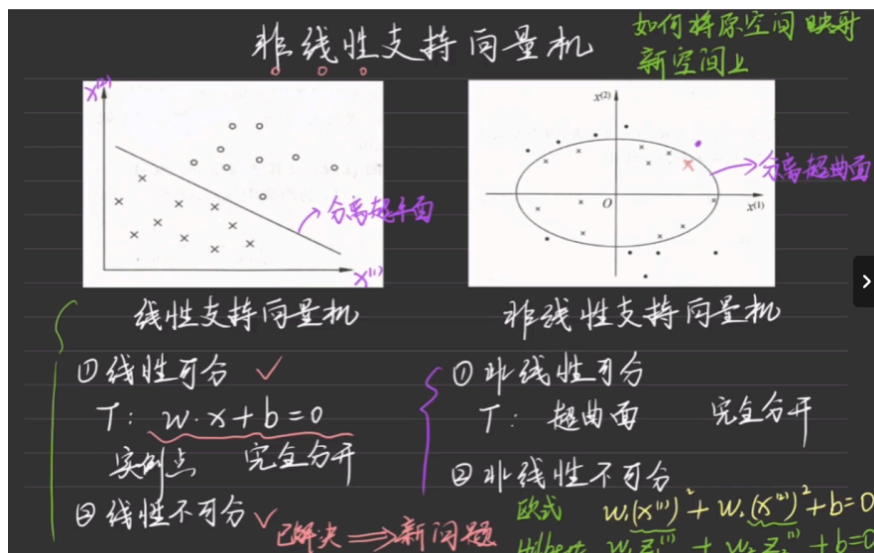
合页损失



注意观察损失函数，其中 $[]_+$ 是合页函数，所以相当于是一个正则化的过程。

非线性支持向量机

线性与非线性的比较：



将非线性转化为希尔伯特空间中，得到一个线性函数，关键在于如何进行转化。

核函数

线性可分支持向量机

$$\min \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N d_i d_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^N d_i$$

原空间: 输入空间 x

新空间: z $z_i \cdot z_j$

st. $\sum_{i=1}^N d_i y_i = 0$ 内积

$d_i \geq 0, i=1, 2, \dots, N$

欧式空间 $x_i \in \mathcal{X}$

希尔伯特空间 $z_i \in \mathcal{H}$

$\phi(x): \mathcal{X} \rightarrow \mathcal{H}$

$z_i = \phi(x_i), z_j = \phi(x_j)$

$z_i \cdot z_j = \phi(x_i) \cdot \phi(x_j) = K(x_i, x_j)$

注意观察 (x_i, x_j) 作为目标函数中的内积，在非线性的中可以转化为两个点在希尔伯特空间中的内积，定义这样的内积问核函数 $K(x_i, x_j)$ 。因此我们不必关心映射到底是什么，而是着重关注核函数的形式。由于原来是内积，因此最后得到核函数的矩阵应该是对称的（半正定矩阵）。
补充说明：变换到希尔伯特空间可以认为是提升向量的维度，使低维不可分的点在高维中变的可分。

如何找正定核? ✓

原: $x_i \cdot x_j$ 新: $\phi(x_i) \cdot \phi(x_j) = K(x_i, x_j)$ 核函数

① $K(x, z)$ 对称函数

$x, z \in \mathcal{X}$ 输入空间

要求: 对 $\forall x_1, \dots, x_m \in \mathcal{X}$, $K(x, z)$ 对应的 Gram 矩阵半正定

原: $\begin{bmatrix} x_1 \cdot x_1 & \dots & x_1 \cdot x_m \\ \vdots & \ddots & \vdots \\ x_m \cdot x_1 & \dots & x_m \cdot x_m \end{bmatrix}_{m \times m}$ 新: $\begin{bmatrix} K(x_1, x_1) & \dots & K(x_1, x_m) \\ \vdots & \ddots & \vdots \\ K(x_m, x_1) & \dots & K(x_m, x_m) \end{bmatrix}_m$

常用核函数:

- 欧式空间下:

常用核函数

1. 欧式空间

① 多项式核函数

$$K(x, z) = (x \cdot z + 1)^p$$

(x \cdot z + 1)^p
↑
常量

决策:

$$f(x) = \text{sign} \left(\sum_{i=1}^N a_i^* y_i (x_i \cdot x + 1)^p + b^* \right)$$

② 高斯核函数

$$K(x, z) = \exp \left(-\frac{\|x - z\|^2}{2\sigma^2} \right)$$

决策:

$$f(x) = \text{sign} \left(\sum_{i=1}^N a_i^* y_i \exp \left(-\frac{\|x - x_i\|^2}{2\sigma^2} \right) + b^* \right)$$

• 离散合集上

考虑使用余弦相似度

例子如下:

$S \geq n \quad s \in S \quad S \xrightarrow{\phi_n(s)} \mathcal{H}_n = \mathbb{R}^n$

$n=2 \quad T \quad S \rightarrow \mathcal{H} = \mathbb{R}^7$

$\Sigma^n = \{ \text{big}, \text{pig}, \text{bag}, \text{big}, \text{pig}, \text{bag}, \text{big}, \text{pig}, \text{bag} \}$

$[\phi_n(s)]_u = \sum_{i: s(i)=u} \lambda^{(i)}$

$\downarrow$
 $-bi$
 $-bg$

$\phi_n(s) \quad K_n(s, t) = \phi_n(s) \cdot \phi_n(t)$

$\text{big} \quad \lambda^2$
 $\text{lass das} \quad \lambda^3$
 $\underline{u=as} = 2\lambda^2 + \lambda^3$
 $\underline{u=asd} = 2\lambda^5$
 $\text{Nasdaq} \quad \lambda^3$

总结

线性支持向量机

输入: 训练集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$
 其中 $x_i \in \mathcal{X} \subseteq \mathbb{R}^d$, $y_i \in \mathcal{Y} \subseteq \{+1, -1\}$
 输出: 分离超平面与分类决策函数

算法:

- 给定惩罚参数 $C > 0$, 构造优化问题

$$\min_w \frac{1}{2} \sum_{i=1}^n d_i \sum_{j=1}^n d_j y_i y_j K(x_i, x_j) - \sum_{i=1}^n d_i$$
 s.t. $\sum_{i=1}^n d_i y_i = 0$
 $0 \leq d_i \leq C$
 $i=1, \dots, N$
- 求解优化问题, 得到最优解 $d^* = (d_1^*, d_2^*, \dots, d_n^*)^T$
- 根据 d^* 得到

$$w^* = \sum_{i=1}^n d_i^* y_i x_i$$
 挑出符合 $0 < d_i^* < C$ 的点 (x_j, y_j) 计算

$$b^* = y_j - \sum_{i=1}^n d_i^* y_i K(x_i, x_j)$$
- 分离超平面: $w^* \cdot x + b^* = 0$
 决策函数: $f(x) = \text{sign}(w^* \cdot x + b^*)$

非线性支持向量机

算法:

- 给定惩罚参数 $C > 0$, 构造优化问题

$$\min_w \frac{1}{2} \sum_{i=1}^n d_i \sum_{j=1}^n d_j y_i y_j K(x_i, x_j) - \sum_{i=1}^n d_i$$
 s.t. $\sum_{i=1}^n d_i y_i = 0$
 $0 \leq d_i \leq C$
 $i=1, \dots, N$
- 求解优化问题, 得到最优解 $d^* = (d_1^*, d_2^*, \dots, d_n^*)^T$
- 根据 d^* 得到 $\psi: x_i \rightarrow K(\cdot, x_i)$

$$w^* = \sum_{i=1}^n d_i^* y_i K(\cdot, x_i)$$
 挑出符合 $0 < d_i^* < C$ 的点 (x_j, y_j) 计算

$$b^* = y_j - \sum_{i=1}^n d_i^* y_i K(x_i, x_j)$$
- 决策函数

$$f(x) = \text{sign}\left(\sum_{i=1}^n d_i^* y_i K(x, x_i) + b^*\right)$$

接下来是具体的优化问题解决方案:

坐标下降法

优化问题

$$\min_w \frac{1}{2} \sum_{i=1}^n d_i \sum_{j=1}^n d_j y_i y_j K(x_i, x_j) - \sum_{i=1}^n d_i$$

s.t. $\sum_{i=1}^n d_i y_i = 0$
 $0 \leq d_i \leq C$
 $i=1, \dots, N$

参数: $d = (d_1, d_2, \dots, d_n)^T$
 $(x_1, y_1) \dots (x_n, y_n)$

N 维变量

1. 坐标下降法: 每次更新一个参数

$$\arg \min_{x_1, x_2} f(x_1, x_2) = x_1^2 + 2x_2^2 - x_1 x_2 + 1$$

① 初始值 $(x_1^{(0)}, x_2^{(0)})^T$

② 固定 $x_2 = x_2^{(0)}$, 求解 x_1
 使得 $\arg \min_{x_1} f(x_1, x_2^{(0)})$

$$\frac{\partial f(x_1, x_2^{(0)})}{\partial x_1} = 2x_1 - x_2^{(0)} = 0$$

$$\Rightarrow x_1^{(1)} = \frac{x_2^{(0)}}{2}$$

③ 固定 $x_1 = x_1^{(1)}$, 求解 x_2
 使得 $\arg \min_{x_2} f(x_1^{(1)}, x_2)$

$$\frac{\partial f(x_1^{(1)}, x_2)}{\partial x_2} = 4x_2 - x_1^{(1)} = 0$$

$$\Rightarrow x_2^{(1)} = \frac{x_1^{(1)}}{4}$$

④ 重复 ②-③ 直到收敛

但是无法用于线性支持向量机, 因此提出了新的方法。

SMO 序列最小最优化算法

$$\begin{aligned} \min_{\alpha} & \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j K(x_i, x_j) - \sum_{i=1}^N \alpha_i \\ \text{s.t.} & \sum_{i=1}^N \alpha_i y_i = 0 \\ & 0 \leq \alpha_i \leq C \\ & i=1, \dots, N \end{aligned} \quad \text{优化问题}$$

3. SMO 序列最小最优化算法

Platt 1998

$$\begin{aligned} \arg \min_{\alpha_1, \alpha_2} W(\alpha_1, \alpha_2) &= -(\alpha_1 + \alpha_2) + \frac{1}{2} \alpha_1^2 y_1^2 K(x_1, x_1) + \frac{1}{2} \alpha_2^2 y_2^2 K(x_2, x_2) \\ &+ \frac{1}{2} \alpha_1 \alpha_2 y_1 y_2 K(x_1, x_2) + \frac{1}{2} \alpha_1 \sum_{j=3}^N \alpha_j y_1 y_j K(x_1, x_j) \\ &+ \frac{1}{2} \alpha_2 \sum_{j=3}^N \alpha_j y_2 y_j K(x_2, x_j) \\ V_1 &= \sum_{j=3}^N \alpha_j y_j K_{1j} \\ V_2 &= \sum_{j=3}^N \alpha_j y_j K_{2j} \\ &= -(\alpha_1 + \alpha_2) + \frac{1}{2} \alpha_1^2 K_{11} + \frac{1}{2} \alpha_2^2 K_{22} + \alpha_1 \alpha_2 y_1 y_2 K_{12} + \alpha_1 V_1 + \alpha_2 V_2 \end{aligned}$$

不同于坐标下降法一次只将一个参数作为变量，SMO一次将两个参数作为变量进行考察。

上式中可以明显看到 α_1 可以用 α_2 表示出来，即 $\alpha_1 = (\sum_{i=3}^N \alpha_i * y_i - \alpha_2 * y_2) * y_1$

带入 w 中可以得到：

$$\begin{aligned} \arg \min_{\alpha_2} W(\alpha_2) &= \left(\frac{1}{2} K_{11} + \frac{1}{2} K_{22} - K_{12} \right) \alpha_2^2 + (y_1 y_2 - y_1^2 K_{12} \xi + K_{12} \xi y_2 \\ &- V_1 y_2 + V_2 y_1) \alpha_2 + (-y_1 \xi + \frac{1}{2} \xi^2 K_{11} + V_1 \xi) \\ \frac{\partial W(\alpha_2)}{\partial \alpha_2} &= 2 \left(\frac{1}{2} K_{11} + \frac{1}{2} K_{22} - K_{12} \right) \alpha_2 + y_2 (y_1 - y_2 + (K_{12} - K_{11}) \xi \\ &+ V_2 - V_1) \\ &= 0 \\ \Rightarrow \hat{\alpha}_2 &= \frac{1}{K_{11} + K_{22} - 2K_{12}} \left[y_2 - y_1 + (K_{11} - K_{12}) \xi + V_1 - V_2 \right] y_2 \end{aligned}$$

$$\begin{aligned} y &= K_{11} + K_{22} - 2K_{12} & K_{11} \xi + V_1 &= \alpha_1 y_1 K_{11} + \alpha_2 y_2 K_{11} \\ K_{12} \xi + V_2 &= \sum_{j=3}^N \alpha_j y_j K_{1j} + (\alpha_1 y_1 K_{12} + \alpha_2 y_2 K_{12} - \alpha_2 y_2 K_{12}) & \alpha_1 y_1 + \alpha_2 y_2 &= \xi \\ &= \sum_{j=3}^N \alpha_j y_j K_{1j} + (\alpha_2 y_2 K_{11} - \alpha_2 y_2 K_{12}) & V_1 &= \sum_{j=3}^N \alpha_j y_j K_{1j} \\ & & V_2 &= \sum_{j=3}^N \alpha_j y_j K_{2j} \end{aligned}$$

此时对 α_2 求导就可以得出对应的结果。

经过一系列化简后可以得到：

最后对算法进行总结可以得到:

SMD 算法

输入:
训练集: $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$
其中 $x_i \in X \in \mathbb{R}^d$, $y_i \in Y \in \{+1, -1\}$
迭代精度: ε

输出: α

算法:

- (1) 取初始值 $\alpha^{(0)} = 0$, 令 $k = 0$;
- (2) 记第 k 次更新之后的参数为 $\alpha^{(k)}$;
- (3) 遍历所有训练样本, 选出违反 KKT 条件的变量 $\alpha^{(k)}$;
- (4) 根据 $|E_i - E_{i_0}|$, 选择差值最大

的变量下标, 记为 $\alpha_{i_0}^{(k)}$;

(5) 通过迭代公式更新参数为 $\alpha_{i_0}^{(k+1)}, \alpha_{j_0}^{(k+1)}$

(6) 若所有的参数在精度 ε 满足

$$\sum_{i=1}^n \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C$$

$y_i g(x_i) \begin{cases} \geq 1 & \alpha_i = 0 \\ = 1 & 0 < \alpha_i < C \\ \leq 1 & \alpha_i = C \end{cases}$

则停止迭代, 否则令 $k = k + 1$ 转 (2)